

TỔNG HỢP KIẾN THỨC TYPESCRIPT & REACTJS

Tài liệu học và ôn tập bằng tiếng Việt - từ nền tảng đến nâng cao

Phần I - TypeScript: ES6+, static typing, compiler, types, functions, OOP, interfaces, generics, advanced types, decorators, modules, namespaces, validation...

Phần II - ReactJS: JSX, components, styling, state, events, effects, forms, lists/keys, router, SSR, Context, RESTful API, middleware, Redux Toolkit...

Cách dùng tài liệu: Đọc theo thứ tự nếu mới học. Nếu đang ôn thi/phỏng vấn, có thể đi thẳng đến các bảng so sánh, ví dụ code và phần checklist cuối tài liệu.

Quy ước: Thuật ngữ kỹ thuật, tên API và từ khóa ngôn ngữ được giữ bằng tiếng Anh để bạn dễ đối chiếu với tài liệu chính thức và code thực tế.

MỤC LỤC NỘI DUNG

PHẦN I - TYPESCRIPT

1. Tổng quan TypeScript và static typing
2. ES6+ nền tảng cần biết
3. Compiler, tsconfig và quy trình biên dịch
4. Basic data types
5. Functions
6. Object types, interfaces và type aliases
7. Classes và OOP
8. Narrowing, type guards và control-flow analysis
9. Advanced types
10. Generics
11. Utility types và thao tác trên type
12. Decorators
13. Modules và namespaces
14. Async programming và error handling
15. Validation trong TypeScript
16. Best practices và lỗi thường gặp

PHẦN II - REACTJS

17. Mental model của React và JSX
18. Components, props và composition
19. Styling
20. State và events
21. Side effects và useEffect
22. Lists và keys
23. React forms
24. Context API và useReducer
25. Custom hooks
26. RESTful API trong React
27. Routing
28. SSR và hydration
29. Performance cơ bản
30. Redux Toolkit, middleware và async flow
31. Kiến trúc ứng dụng, error handling và best practices
32. Cheat sheet + checklist ôn tập

PHẦN I - TYPESCRIPT

JavaScript có hệ thống kiểu tĩnh, nhưng vẫn chạy trên hệ sinh thái JavaScript. TypeScript (TS) là superset của JavaScript: hầu hết JavaScript hợp lệ cũng là TypeScript hợp lệ. TypeScript bổ sung hệ thống type, khả năng phân tích tĩnh và tooling tốt hơn, sau đó được compiler chuyển thành JavaScript để chạy trong browser, Node.js hoặc môi trường JavaScript khác.

Mục tiêu: Không phải “thêm type cho đẹp”, mà là làm cho contract của dữ liệu, function và module rõ ràng hơn để lỗi được phát hiện sớm trước runtime.

1. Tổng quan TypeScript và Static Typing

1.1 Static typing là gì?

Static typing nghĩa là nhiều lỗi liên quan đến kiểu dữ liệu được kiểm tra trước khi chương trình chạy. TypeScript dùng type inference để tự suy luận type, nên bạn không cần annotation ở mọi nơi.

```
let username = "An";      // TS suy luận: string
// username = 123;       // Error: number không gán được cho string

const age: number = 21;
const isStudent: boolean = true;
```

Khái niệm	Ý nghĩa
Type annotation	Bạn khai báo type trực tiếp: <code>const n: number = 10</code> .
Type inference	Compiler tự suy luận type từ giá trị/ngữ cảnh.
Type checking	Kiểm tra tính tương thích giữa các type.
Runtime	Sau khi compile, type của TS phần lớn không còn tồn tại trong JavaScript output.

1.2 Structural typing

TypeScript chủ yếu dùng structural typing: hai object được xem là tương thích nếu cấu trúc cần thiết tương thích, không nhất thiết phải được tạo từ cùng một class/interface.

```
type Point = { x: number; y: number };

const p3d = { x: 1, y: 2, z: 3 };
const p2d: Point = p3d; // hợp lệ vì có ít nhất x và y đúng type
```

Ghi nhớ: TypeScript giúp kiểm tra compile-time; nếu dữ liệu đến từ API, localStorage hoặc user input thì vẫn cần runtime validation.

2. ES6+ nền tảng cần biết

2.1 let, const và scope

- `const`: binding không thể gán lại; object/array bên trong vẫn có thể thay đổi nếu không `readonly/freeze`.
- `let`: dùng khi biến cần gán lại.
- Tránh `var` vì function scope và hoisting dễ gây lỗi khó đọc.

```
const user = { name: "Lan" };
user.name = "Mai"; // hợp lệ

let count = 0;
count += 1;
```

2.2 Destructuring, spread và rest

```
const profile = { id: 1, name: "Minh", role: "admin" };
const { name, role } = profile;

const updated = { ...profile, role: "editor" };

function sum(...numbers: number[]) {
  return numbers.reduce((total, n) => total + n, 0);
}
```

2.3 Arrow functions và lexical this

Arrow function ngắn gọn và không tạo `this` riêng. Điều này hữu ích trong callback, nhưng method phụ thuộc dynamic `this` đôi khi nên dùng function/method thông thường.

```
const double = (n: number) => n * 2;

class Timer {
  seconds = 0;
  start() {
    setInterval(() => {
      this.seconds++; // this là instance Timer
    }, 1000);
  }
}
```

2.4 Template literals, optional chaining, nullish coalescing

```
const label = `${profile.name} - ${profile.role}`;
const city = userAddress?.city;
const pageSize = config.pageSize ?? 20; // chỉ fallback khi null/undefined
```

2.5 Promise và async/await

```
async function loadUser(id: number) {
  try {
    const response = await fetch(`/api/users/${id}`);
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return await response.json();
  } catch (error) {
    console.error(error);
    throw error;
  }
}
```

2.6 Map, Set và module syntax

```
const roles = new Set(["admin", "editor"]);
const cache = new Map<number, string>();
cache.set(1, "Alice");

export const API_URL = "/api";
export default function App() {}
// import App, { API_URL } from "./app";
```

3. Compiler, tsconfig và quy trình biên dịch

3.1 Từ .ts/.tsx thành JavaScript

TypeScript compiler (`tsc`) đọc source code, type-check và emit JavaScript tùy cấu hình. Trong dự án frontend hiện đại, bundler/build tool thường xử lý transpile; tuy nhiên type-check vẫn nên được chạy độc lập trong CI.

```
# Khởi tạo cấu hình
tsc --init

# Kiểm tra/biên dịch project
tsc

# Chỉ kiểm tra type, không emit JS
tsc --noEmit
```

3.2 Các lựa chọn tsconfig quan trọng

Option	Tác dụng thực tế
strict	Bật nhóm kiểm tra chặt; nên ưu tiên trong project mới.
target	Mức JavaScript đầu ra mà môi trường runtime hỗ trợ.
module	Cách module được emit/hiểu.
moduleResolution	Quy tắc tìm module/package.
jsx	Cách xử lý JSX trong file TSX.
esModuleInterop	Tăng khả năng tương thích import CommonJS.
noUncheckedIndexedAccess	Truy cập index có thể trả về undefined; giúp an toàn hơn.
noEmit	Chỉ type-check, thường dùng khi bundler đảm nhiệm build.

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "strict": true,
    "jsx": "react-jsx",
    "noUncheckedIndexedAccess": true,
    "noEmit": true
  },
  "include": ["src"]
}
```

Lưu ý: `target`, `module` và `moduleResolution` nên phù hợp build tool/runtime của dự án. Không nên copy tsconfig giữa các project một cách máy móc.

4. Basic Data Types

4.1 Primitive types

```
let title: string = "TypeScript";
let score: number = 9.5;
let active: boolean = true;
let big: bigint = 9007199254740993n;
let key: symbol = Symbol("key");
let empty: null = null;
let missing: undefined = undefined;
```

4.2 Array, tuple và readonly

```
const ids: number[] = [1, 2, 3];
const names: Array<string> = ["A", "B"];

type HttpResult = [status: number, body: string];
const result: HttpResult = [200, "OK"];

const coordinates: readonly [number, number] = [10, 20];
```

4.3 enum và literal union

`enum` tồn tại trong TypeScript, nhưng nhiều project thích literal union vì output đơn giản và dễ interoperable với JSON.

```
type Role = "admin" | "editor" | "viewer";
let role: Role = "admin";

enum Direction { Up, Down, Left, Right }
```

4.4 any, unknown, never, void

Type	Khi dùng
any	Tất gần như toàn bộ type safety ở vị trí đó. Chỉ dùng khi thật sự cần escape hatch.
unknown	Giá trị chưa biết type; phải narrow trước khi thao tác. An toàn hơn any.
never	Không có giá trị hợp lệ: function luôn throw/loop, hoặc exhaustive checking.
void	Function không trả về giá trị có ý nghĩa.

```
function printValue(value: unknown) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  }
}

function fail(message: string): never {
  throw new Error(message);
}
```

5. Functions

5.1 Parameter, return type và optional/default

```
function greet(name: string, prefix = "Hello"): string {
  return `${prefix}, ${name}`;
}

function findUser(id: number, includeDeleted?: boolean) {
  // includeDeleted có type boolean | undefined
}
```

5.2 Function type và callback

```
type Comparator<T> = (a: T, b: T) => number;

const compareNumber: Comparator<number> = (a, b) => a - b;
[3, 1, 2].sort(compareNumber);
```

5.3 Overloads

Overload hữu ích khi một function có nhiều call signature rõ ràng. Implementation signature phải bao phủ các overload.

```
function format(value: number): string;
function format(value: Date): string;
function format(value: number | Date): string {
  return value instanceof Date ? value.toISOString() : value.toFixed(2);
}

format(3.14159);
format(new Date());
```

5.4 this parameter

```
interface Counter { value: number }

function increment(this: Counter, amount: number) {
  this.value += amount;
}

const counter: Counter = { value: 0 };
increment.call(counter, 2);
```

6. Object Types, Interfaces và Type Aliases

6.1 Object type

```
type User = {
  readonly id: number;
  name: string;
  email?: string;
};

const user: User = { id: 1, name: "Nam" };
```

6.2 interface

```
interface Person {
  name: string;
  age: number;
}

interface Employee extends Person {
  employeeId: string;
}

const e: Employee = { name: "Hoa", age: 22, employeeId: "E01" };
```

6.3 interface vs type

interface	type
Tốt cho shape của object/class contract.	Linh hoạt cho union, intersection, tuple, primitive alias...
Có declaration merging.	Không declaration merging cùng tên.

Có extends.	Có thể compose bằng intersection (&).
Rất phù hợp public object API.	Phù hợp các phép biến đổi type phức tạp.

Trong thực tế, cả hai đều tốt. Quan trọng nhất là convention nhất quán của codebase và chọn công cụ diễn đạt type rõ nhất.

7. Classes và OOP

7.1 Class cơ bản

```
class BankAccount {
  private balance = 0;

  constructor(
    public readonly owner: string,
    protected currency: string = "VND"
  ) {}

  deposit(amount: number): void {
    if (amount <= 0) throw new Error("Amount must be positive");
    this.balance += amount;
  }

  getBalance(): number {
    return this.balance;
  }
}

const account = new BankAccount("An");
account.deposit(100_000);
```

7.2 public, private, protected, readonly

- ``public``: mặc định; truy cập bên ngoài class.
- ``private``: chỉ class khai báo truy cập được theo kiểm tra của TypeScript.
- ``protected``: class và subclass truy cập được.
- ``readonly``: chỉ được gán trong initialization/constructor theo luật của TypeScript.

7.3 abstract class và implements

```
interface Serializable {
  serialize(): string;
}

abstract class Shape {
  abstract area(): number;
}

class Rectangle extends Shape implements Serializable {
  constructor(private w: number, private h: number) { super(); }
  area() { return this.w * this.h; }
  serialize() { return JSON.stringify({ w: this.w, h: this.h }); }
}
```

Thiết kế: Trong frontend React, composition thường được ưu tiên hơn inheritance. Class vẫn hữu ích cho domain model, service đặc thù hoặc thư viện, nhưng không nên ép OOP vào mọi bài toán.

8. Narrowing, Type Guards và Control-flow Analysis

8.1 typeof, instanceof, in

```
function normalize(value: string | number) {
  if (typeof value === "string") return value.trim();
  return value.toFixed(0);
}

function getMessage(error: unknown) {
  if (error instanceof Error) return error.message;
  return String(error);
}

type Admin = { role: "admin"; permissions: string[] };
type Guest = { role: "guest" };
```

```
function canManage(u: Admin | Guest) {
  return "permissions" in u;
}
```

8.2 Discriminated union

```
type RequestState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };

function renderState<T>(state: RequestState<T>) {
  switch (state.status) {
    case "success": return state.data;
    case "error": return state.error;
    default: return null;
  }
}
```

8.3 Custom type predicate

```
type User = { id: number; name: string };

function isUser(value: unknown): value is User {
  if (typeof value !== "object" || value === null) return false;
  const obj = value as Record<string, unknown>;
  return typeof obj.id === "number" && typeof obj.name === "string";
}
```

9. Advanced Types

9.1 Union, intersection và literal types

```
type Id = string | number;
type Timestamped = { createdAt: Date };
type Named = { name: string };
type Entity = Timestamped & Named;

type Method = "GET" | "POST" | "PUT" | "PATCH" | "DELETE";
```

9.2 keyof, typeof và indexed access

```
const settings = { theme: "dark", pageSize: 20 };
type Settings = typeof settings;
type SettingsKey = keyof Settings; // "theme" | "pageSize"
type ThemeType = Settings["theme"]; // string

function get<T, K extends keyof T>(obj: T, key: K): T[K] {
  return obj[key];
}
```

9.3 Conditional types và infer

```
type ElementType<T> = T extends Array<infer U> ? U : T;
type A = ElementType<string[]>; // string
type B = ElementType<number>; // number

type ApiResult<T> = T extends Error
  ? { ok: false; error: T }
  : { ok: true; data: T };
```

9.4 Mapped types

```
type Flags<T> = {
  [K in keyof T]: boolean;
};

type User = { id: number; name: string };
type UserFlags = Flags<User>; // { id: boolean; name: boolean }
```

9.5 Template literal types

```
type EventName = "click" | "change";
type HandlerName = `on${Capitalize<EventName>}`;
// "onClick" | "onChange"
```


9.6 satisfies và as const

```
const routes = {
  home: "/",
  users: "/users"
} as const;

type RouteName = keyof typeof routes;

const config = { mode: "production", retry: 3 } satisfies {
  mode: "development" | "production";
  retry: number;
};
```

Tư duy: Advanced types nên làm type dễ dùng hơn, không phải biến type system thành câu đố. Nếu một type mất nhiều phút mới hiểu, cân nhắc đơn giản hóa API.

10. Generics

10.1 Generic function

```
function identity<T>(value: T): T {
  return value;
}

const n = identity(123);    // T = number
const s = identity("hello"); // T = string
```

10.2 Constraint

```
type WithId = { id: string | number };

function indexById<T extends WithId>(items: T[]): Map<T["id"], T> {
  return new Map(items.map(item => [item.id, item]));
}
```

10.3 Generic interface/class

```
interface Repository<T, ID = number> {
  findById(id: ID): Promise<T | null>;
  save(entity: T): Promise<T>;
}

class MemoryStore<T> {
  private data: T[] = [];
  add(item: T) { this.data.push(item); }
  all(): readonly T[] { return this.data; }
}
```

10.4 Generic trong React

```
type SelectProps<T> = {
  items: T[];
  getLabel: (item: T) => string;
  onSelect: (item: T) => void;
};

function Select<T>({ items, getLabel, onSelect }: SelectProps<T>) {
  return (
    <ul>
      {items.map((item, index) => (
        <li key={index} onClick={() => onSelect(item)}>
          {getLabel(item)}
        </li>
      ))}
    </ul>
  );
}
```

Lưu ý React: Ví dụ generic component trên dùng index key chỉ để minh họa type. Trong ứng dụng thật, nên dùng key ổn định từ dữ liệu nếu danh sách có thể thay đổi/reorder.

11. Utility Types và thao tác trên Type

Utility type	Ý nghĩa
--------------	---------

Partial<T>	Mọi property thành optional.
Required<T>	Mọi property thành required.
Readonly<T>	Mọi property readonly.
Pick<T, K>	Chọn một tập key.
Omit<T, K>	Bỏ một tập key.
Record<K, V>	Object có key K và value V.
Exclude<U, M>	Loại member M khỏi union U.
Extract<U, M>	Lấy member tương thích M từ union U.
Nullable<T>	Loại null và undefined.
ReturnType<F>	Lấy return type của function type.
Parameters<F>	Lấy tuple parameter types.
Awaited<T>	Unwrap Promise-like type.

```
type User = {
  id: number;
  name: string;
  email: string;
  passwordHash: string;
};

type UserUpdate = Partial<Pick<User, "name" | "email">>;
type PublicUser = Omit<User, "passwordHash">;
type UserId = Record<number, PublicUser>;
```

12. Decorators

Decorator cho phép gắn logic/metadata quanh class hoặc member. Cần phân biệt decorator theo chuẩn JavaScript mới và “legacy decorators” từng phổ biến trong TypeScript. Cú pháp/cấu hình cụ thể phụ thuộc môi trường/framework, vì vậy hãy theo convention của project.

12.1 Ví dụ class decorator dạng khái niệm

```
function sealed<T extends Function>(target: T) {
  Object.seal(target);
  Object.seal(target.prototype);
}

@sealed
class Service {
  run() { return "ok"; }
}
```

12.2 Khi nào decorators hữu ích?

- Dependency injection / IoC container.
- ORM mapping, serialization, validation metadata.
- Framework dùng annotation-style API.
- Cross-cutting concern như logging/authorization - nếu framework hỗ trợ rõ ràng.

Cảnh báo: Decorator có thể làm control flow ẩn và tăng “magic”. Với code ứng dụng đơn giản, function composition thường dễ đọc/test hơn.

13. Modules và Namespaces

13.1 ES modules là lựa chọn mặc định

```
// math.ts
export function add(a: number, b: number) { return a + b; }
export const PI = 3.14159;

// app.ts
import { add, PI } from "./math";
```

13.2 Type-only import

```
import type { User } from "./types";

export function printUser(user: User) {
  console.log(user.name);
}
```

13.3 Namespace

`namespace` là cơ chế tổ chức code của TypeScript trước khi ES modules trở thành chuẩn phổ biến. Code ứng dụng hiện đại thường ưu tiên ES modules. Namespace vẫn xuất hiện trong legacy code hoặc declaration đặc thù.

```
namespace Validation {
  export function isEmail(value: string) {
    return value.includes("@");
  }
}

Validation.isEmail("a@example.com");
```

13.4 Declaration files (.d.ts)

File `.d.ts` mô tả type cho JavaScript/library mà không tạo runtime code. Bạn thường gặp chúng trong package type definitions hoặc khi khai báo module/global đặc biệt.

```
declare module "legacy-lib" {
  export function parse(input: string): unknown;
}
```

14. Async Programming và Error Handling

14.1 Promise type

```
type User = { id: number; name: string };

async function fetchUser(id: number): Promise<User> {
  const res = await fetch(`/api/users/${id}`);
  if (!res.ok) throw new Error(`Request failed: ${res.status}`);
  return res.json() as Promise<User>;
}
```

Quan trọng: `as Promise<User>` chỉ là assertion compile-time; nó không xác minh JSON thật sự có shape User. Dữ liệu external cần validation nếu độ tin cậy quan trọng.

14.2 Result pattern

```
type Result<T, E = Error> =
  | { ok: true; value: T }
  | { ok: false; error: E };

async function safeFetch<T>(url: string): Promise<Result<T>> {
  try {
    const res = await fetch(url);
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return { ok: true, value: await res.json() as T };
  } catch (error) {
    return {
      ok: false,
      error: error instanceof Error ? error : new Error(String(error))
    };
  }
}
```

14.3 AbortController

```
const controller = new AbortController();
fetch("/api/search?q=react", { signal: controller.signal });
controller.abort();
```

15. Validation trong TypeScript

15.1 Vì sao cần runtime validation?

TypeScript type bị xóa khi runtime. JSON từ API, `JSON.parse`, query string, form input hoặc localStorage đều là dữ liệu runtime và có thể không đúng type bạn mong đợi.

15.2 Manual validator/type guard

```
type Product = {
  id: number;
```

```

    name: string;
    price: number;
  };

  function isProduct(value: unknown): value is Product {
    if (typeof value !== "object" || value === null) return false;
    const v = value as Record<string, unknown>;
    return (
      typeof v.id === "number" &&
      typeof v.name === "string" &&
      typeof v.price === "number"
    );
  }

  const parsed: unknown = JSON.parse(rawJson);
  if (!isProduct(parsed)) throw new Error("Invalid product payload");

```

15.3 Schema validator (ví dụ với Zod)

```

import { z } from "zod";

const ProductSchema = z.object({
  id: z.number(),
  name: z.string().min(1),
  price: z.number().nonnegative()
});

type Product = z.infer<typeof ProductSchema>;
const product = ProductSchema.parse(JSON.parse(rawJson));

```

- Schema validator phù hợp khi payload lớn, nhiều rule hoặc cần error detail.
- Manual type guard phù hợp validation nhỏ, không muốn thêm dependency.
- Không nên dùng type assertion (`as User`) để thay thế validation.

16. TypeScript Best Practices và lỗi thường gặp

Nên	Tránh
Bật strict và sửa type errors có chủ đích.	Tắt strict để "code chạy trước".
Ưu tiên inference khi type đã rõ.	Annotate mọi biến làm code nặng nề.
Dùng unknown ở boundary không tin cậy.	Dùng any lan rộng.
Model state bằng discriminated union.	Nhiều boolean có thể tạo trạng thái vô lý.
Dùng type guard/schema ở external boundary.	Ép `as` dữ liệu API rồi tin tuyệt đối.
Giữ generic/type helper đơn giản.	Type-level programming quá phức tạp.
Dùng readonly cho dữ liệu không nên mutate.	Mutate shared state tùy ý.

16.1 Exhaustive checking

```

type Status = "draft" | "published" | "archived";

function label(status: Status): string {
  switch (status) {
    case "draft": return "Nháp";
    case "published": return "Đã xuất bản";
    case "archived": return "Lưu trữ";
    default: {
      const _never: never = status;
      return _never;
    }
  }
}

```

16.2 TypeScript checklist

- Biết phân biệt compile-time và runtime.
- Biết `any` khác `unknown` như thế nào.
- Biết union + narrowing + discriminated union.
- Biết interface/type, class/implements, readonly/optional.
- Biết generic + constraint + keyof.
- Biết Partial/Pick/Omit/Record/ReturnType.
- Biết module/import/export và type-only import.
- Biết validation boundary cho API/form/storage.

PHẦN II - REACTJS

Xây dựng UI theo component, state và data flow khai báo (declarative)

React là thư viện xây dựng giao diện dựa trên component. Thay vì thao tác DOM thủ công theo từng bước, bạn mô tả UI dựa trên props/state; React chịu trách nhiệm đồng bộ giao diện với state hiện tại.

Mental model: Render là phép tính từ props/state sang UI. Side effect là những việc tương tác với thế giới bên ngoài render: network, timer, subscription, DOM API đặc biệt, storage...

17. JSX và Mental Model của React

17.1 JSX

JSX là cú pháp gần giống HTML nhưng nằm trong JavaScript/TypeScript. Trong TypeScript, file có JSX thường dùng đuôi `.tsx`. Expression JavaScript được đặt trong `{}`.

```
type GreetingProps = { name: string; unread: number };

function Greeting({ name, unread }: GreetingProps) {
  return (
    <section className="card">
      <h2>Xin chào, {name}</h2>
      {unread > 0 && <p>Bạn có {unread} thông báo mới.</p>}
    </section>
  );
}
```

17.2 Một số khác biệt với HTML

HTML	JSX
class	className
for	htmlFor
onclick	onClick
style="color:red"	style={{ color: "red" }}
Thuộc tính dạng string	Nhiều prop nhận expression JS trong {}

17.3 Render phải “pure” về mặt side effect

- Không gọi API trực tiếp trong thân component mỗi lần render.
- Không mutate state trong render.
- Có thể tính derived data từ props/state trong render.
- Event handler và effect là nơi thực hiện các hành động thay đổi thế giới bên ngoài.

18. Components, Props và Composition

18.1 Function component

```
type AvatarProps = {
  src: string;
  alt: string;
  size?: number;
};

function Avatar({ src, alt, size = 48 }: AvatarProps) {
  return <img src={src} alt={alt} width={size} height={size} />;
}
```

18.2 children và composition

```
import type { ReactNode } from "react";

type CardProps = {
  title: string;
  children: ReactNode;
};

function Card({ title, children }: CardProps) {
```

```

return (
  <section className="card">
    <h3>{title}</h3>
    <div>{children}</div>
  </section>
);
}

```

18.3 Props là read-only input

Component không nên sửa trực tiếp props. Nếu dữ liệu cần thay đổi, state thường nằm ở component sở hữu dữ liệu, rồi truyền xuống cùng callback để component con yêu cầu thay đổi.

18.4 Lifting state up

Khi hai component cần dùng chung một state, đưa state lên ancestor gần nhất có thể sở hữu source of truth, sau đó truyền dữ liệu và event callback xuống.

19. Styling trong React

Cách	Đặc điểm
CSS thường	Đơn giản, global scope; cần naming convention.
CSS Modules	Class name scoped theo module, giảm xung đột.
Inline style	Tiện cho style tính toán động; không phù hợp mọi rule như pseudo-selector.
CSS-in-JS	Style gắn với component; phụ thuộc library/chỉ phí runtime hoặc build.
Utility CSS	Kết hợp utility class; nhanh và nhất quán nếu team theo convention.

```

// CSS Module
import styles from "./Button.module.css";

function Button() {
  return <button className={styles.primary}>Luu</button>;
}

// Inline style
<div style={{ opacity: disabled ? 0.5 : 1 }} />

```

Best practice: Chọn 1-2 chiến lược styling chủ đạo cho project. Sự nhất quán quan trọng hơn việc tranh luận “cách nào tuyệt đối tốt nhất”.

20. State và Events

20.1 useState

```

import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(c => c + 1)}>
      Count: {count}
    </button>
  );
}

```

20.2 State update bất đồng bộ theo batch và functional update

Khi state mới phụ thuộc state cũ, dùng updater function (`setCount(c => c + 1)`) để tránh closure giữ giá trị cũ trong các tình huống update liên tiếp.

20.3 Object/array state phải update bất biến

```

type Todo = { id: number; text: string; done: boolean };

const [todos, setTodos] = useState<Todo[]>([]);

function toggle(id: number) {

```

```

    setTodos(items =>
      items.map(item =>
        item.id === id ? { ...item, done: !item.done } : item
      )
    );
  }
}

```

20.4 Event typing với TypeScript

```

import type { ChangeEvent, FormEvent } from "react";

function Search() {
  const [q, setQ] = useState("");

  function handleChange(e: ChangeEvent<HTMLInputElement>) {
    setQ(e.target.value);
  }

  function handleSubmit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
  }

  return <form onSubmit={handleSubmit}><input value={q} onChange={handleChange} /></form>;
}

```

21. Side Effects và useEffect

21.1 Effect dùng để đồng bộ với external system

Effect phù hợp cho subscription, timer, network request thủ công, browser API, imperative library... Không nên dùng effect chỉ để tính dữ liệu có thể tính trực tiếp từ props/state.

```

import { useEffect, useState } from "react";

function Clock() {
  const [now, setNow] = useState(() => new Date());

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id); // cleanup
  }, []);

  return <time>{now.toLocaleTimeString()}</time>;
}

```

21.2 Dependency array

Dạng	Ý nghĩa khái quát
Không có dependency array	Effect chạy sau mỗi commit phù hợp điều kiện của React.
[]	Thiết lập/cleanup theo lifecycle mount-unmount trong mô hình phổ biến.
[a, b]	Chạy lại khi dependency liên quan thay đổi.

21.3 Tránh effect không cần thiết

```

// Không cần effect
const fullName = `${firstName} ${lastName}`;

// Tránh kiểu:
// useEffect(() => setFullName(`${firstName} ${lastName}`), [firstName, lastName]);

```

21.4 Fetch và cleanup bằng AbortController

```

useEffect(() => {
  const controller = new AbortController();

  async function load() {
    const res = await fetch(`/api/users/${userId}`, {
      signal: controller.signal
    });
    // xử lý response...
  }

  load().catch(error => {

```

```

    if (error instanceof DOMException && error.name === "AbortError") return;
    console.error(error);
  });

  return () => controller.abort();
}, [userId]);

```

22. Lists và Keys

```

type User = { id: number; name: string };

function UserList({ users }: { users: User[] }) {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}

```

22.1 Key để React nhận diện identity

- Key phải ổn định giữa các render.
- Ưu tiên ID thật từ dữ liệu.
- Không tạo key bằng `Math.random()` trong render.
- Index có thể chấp nhận cho danh sách tĩnh không reorder/insert/delete; nếu danh sách động, index dễ gây state/UI “nhảy” sai item.

22.2 filter + map

```

const activeUsers = users
  .filter(user => user.active)
  .map(user => <UserRow key={user.id} user={user} />);

```

23. React Forms

23.1 Controlled input

```

function LoginForm() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");

  function submit(e: FormEvent<HTMLFormElement>) {
    e.preventDefault();
    console.log({ email, password });
  }

  return (
    <form onSubmit={submit}>
      <input
        type="email"
        value={email}
        onChange={e => setEmail(e.target.value)}
      />
      <input
        type="password"
        value={password}
        onChange={e => setPassword(e.target.value)}
      />
      <button type="submit">Đăng nhập</button>
    </form>
  );
}

```

23.2 Validation

- HTML validation: required, minLength, pattern, type=email... dùng được cho rule đơn giản.
- Custom validation: kiểm tra state và hiển thị error message rõ ràng.
- Schema validation: hữu ích khi form lớn hoặc chia sẻ schema với API/domain.
- Server vẫn phải validate lại; validation client chỉ cải thiện UX, không phải security boundary.

23.3 Form library

Với form phức tạp, thư viện như React Hook Form có thể giảm boilerplate và quản lý validation/touched/error hiệu quả hơn. Tuy vậy, hiểu controlled/uncontrolled input và HTML form trước sẽ giúp bạn dùng library đúng cách.

23.4 Accessibility

```
<label htmlFor="email">Email</label>
<input id="email" name="email" type="email" aria-describedby="email-error" />
<p id="email-error" role="alert">Email không hợp lệ</p>
```

24. Context API và useReducer

24.1 Context giải quyết prop drilling cho dữ liệu dùng rộng

```
import { createContext, useContext } from "react";

type Theme = "light" | "dark";
const ThemeContext = createContext<Theme | null>(null);

function useTheme() {
  const value = useContext(ThemeContext);
  if (value === null) throw new Error("ThemeProvider is missing");
  return value;
}

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Page />
    </ThemeContext.Provider>
  );
}
```

24.2 Không phải mọi state global đều cần Context

- Context phù hợp theme, locale, auth session summary, dependency/config dùng rộng.
- State chỉ vài component dùng thì giữ local/lift up thường đơn giản hơn.
- Context value thay đổi thường xuyên có thể làm nhiều consumer render lại; cần thiết kế scope hợp lý.

24.3 useReducer

```
type State = { count: number };
type Action =
  | { type: "increment" }
  | { type: "add"; amount: number };

function reducer(state: State, action: Action): State {
  switch (action.type) {
    case "increment": return { count: state.count + 1 };
    case "add": return { count: state.count + action.amount };
  }
}

const [state, dispatch] = useReducer(reducer, { count: 0 });
```

`useReducer` hữu ích khi state transition phức tạp, nhiều action liên quan hoặc muốn gom logic update vào một nơi.

25. Custom Hooks

Custom hook là function có tên bắt đầu bằng `use` và có thể dùng các hooks khác. Nó tái sử dụng stateful logic, không phải tái sử dụng UI.

```
import { useEffect, useState } from "react";

function useDebounceValue<T>(value: T, delayMs: number) {
  const [debounced, setDebounce] = useState(value);

  useEffect(() => {
    const id = setTimeout(() => setDebounce(value), delayMs);
    return () => clearTimeout(id);
  }, [value, delayMs]);
}
```

```
    return debounced;
  }
}
```

25.1 Quy tắc thiết kế custom hook

- Tên thể hiện mục đích: `useAuth`, `useDebounceValue`, `useOnlineStatus`...
- API trả về phải dễ hiểu và ổn định.
- Không biến custom hook thành “god hook” xử lý quá nhiều domain.
- Hooks phải tuân thủ Rules of Hooks: gọi ở top-level của component/custom hook, không gọi tùy tiện trong loop/if thông thường.

26. RESTful API trong React

26.1 HTTP request anatomy

Thành phần	Ví dụ
Method	GET, POST, PUT, PATCH, DELETE
URL	/api/users/42
Path params	42 trong /users/42
Query params	?page=2&limit=20
Headers	Content-Type, Authorization, Accept...
Body	JSON cho POST/PUT/PATCH khi phù hợp

26.2 HTTP response anatomy

Thành phần	Ý nghĩa
Status code	200, 201, 204, 400, 401, 403, 404, 409, 422, 500...
Headers	Content-Type, Cache-Control, ETag...
Body	JSON/data/error payload; 204 thường không có body.

26.3 API helper với fetch

```
type ApiError = {
  message: string;
  status: number;
};

async function request<T>(url: string, init?: RequestInit): Promise<T> {
  const res = await fetch(url, {
    ...init,
    headers: {
      "Content-Type": "application/json",
      ...init?.headers
    }
  });

  if (!res.ok) {
    throw {
      message: `Request failed with status ${res.status}`,
      status: res.status
    } satisfies ApiError;
  }

  if (res.status === 204) return undefined as T;
  return res.json() as Promise<T>;
}
```

26.4 Loading, success, empty, error

UI gọi API nên nghĩ theo state machine thay vì chỉ có `data`. Ít nhất cần cần nhắc loading, success, empty và error. Với mutation còn có submitting/pending, success feedback và rollback/refresh.

```
type AsyncState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; message: string };
```

26.5 Authentication token

Không hard-code token. Cơ chế lưu token/cookie phụ thuộc kiến trúc security. Nếu backend dùng cookie session, cần hiểu CORS/credentials/CSRF. Nếu dùng bearer token, cần quản lý refresh/expiry và tránh lộ token qua log hoặc source code.

26.6 API layer vs component

Component nên tập trung vào UI và interaction. Request construction, serialization, error normalization và schema validation có thể tách sang API/service layer hoặc data-fetching library để test và tái sử dụng dễ hơn.

27. Router

27.1 Client-side routing

Router ánh xạ URL vào UI mà không yêu cầu full page reload cho mọi chuyển trang. Các khái niệm chính: route, nested route, params, query/search params, navigation, layout và 404 route.

```
// Ví dụ khái niệm với React Router
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/users">Users</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/users" element={<UserList />} />
        <Route path="/users/:id" element={<UserDetail />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

27.2 Route params và search params

```
// /users/42?tab=posts
const { id } = useParams();
const [searchParams] = useSearchParams();
const tab = searchParams.get("tab");
```

27.3 Protected route

Protected route trên client chỉ là UX/navigation guard; authorization thật sự phải được backend kiểm tra. Không dựa vào việc "ẩn route/button" để bảo mật dữ liệu.

28. SSR và Hydration

28.1 CSR, SSR và SSG

Mô hình	Mô tả
CSR	Browser tải JS rồi render UI chủ yếu phía client.
SSR	Server render HTML cho request, sau đó client hydrate để tương tác.
SSG	HTML được tạo trước ở build/generation time cho nội dung phù hợp.

28.2 Lợi ích và trade-off SSR

- Có thể cải thiện initial HTML, SEO và perceived performance trong nhiều trường hợp.
- Tăng độ phức tạp: server runtime, data loading, cache, hydration mismatch, code chỉ chạy browser.
- Không phải mọi trang đều cần SSR. Dashboard nội bộ có thể phù hợp CSR; trang public/SEO thường hưởng lợi hơn.

28.3 Hydration mismatch

Hydration mismatch xảy ra khi HTML server và render client ban đầu không khớp. Nguồn thường gặp: dùng `Date.now()`, random value, browser-only API hoặc dữ liệu khác nhau ngay trong initial render.

```
// Tránh đọc window trực tiếp trong render server-side
useEffect(() => {
  const width = window.innerWidth;
  // cập nhật state sau khi client mount nếu cần
}, []);
```

Framework: Trong dự án thực tế, SSR thường được quản lý bởi framework React. Hãy theo data-loading, routing, caching và server/client boundary của framework mà project dùng.

29. Performance cơ bản

29.1 Tối ưu đúng chỗ

Đầu tiên hãy tối ưu architecture, tránh request dư, render list khổng lồ và state đặt quá cao. `memo`, `useMemo`, `useCallback` là công cụ tối ưu có chi phí và không nên thêm mặc định khắp nơi.

29.2 memo/useMemo/useCallback

```
const filtered = useMemo(
  () => items.filter(item => item.name.includes(query)),
  [items, query]
);

const handleSelect = useCallback((id: number) => {
  setSelectedId(id);
}, []);
```

29.3 Các vấn đề hay gặp

- State global quá rộng làm nhiều component re-render.
- Context value tạo object mới mỗi render khi không cần.
- Danh sách rất dài không virtualize.
- Effect chạy vòng lặp do dependency thay đổi liên tục.
- Fetch trùng lặp, thiếu cache/deduplication.
- Bundle lớn vì import library nặng hoặc không code-split đúng chỗ.

30. Redux Toolkit, Middleware và Async Flow

30.1 Khi nào cần state manager?

Không phải app nào cũng cần Redux. Redux hữu ích khi client state phức tạp, nhiều phần app cùng đọc/ghi, cần predictable transitions, debugging tốt hoặc business workflow có nhiều action. Server cache/data fetching lại là một bài toán khác và có thể dùng data-fetching abstraction phù hợp.

30.2 Store, slice, reducer, action

```
import { configureStore, createSlice, PayloadAction } from "@reduxjs/toolkit";

type CounterState = { value: number };
const initialState: CounterState = { value: 0 };

const counterSlice = createSlice({
  name: "counter",
  initialState,
  reducers: {
    increment(state) {
      state.value += 1;
    },
    add(state, action: PayloadAction<number>) {
      state.value += action.payload;
    }
  }
});

export const { increment, add } = counterSlice.actions;

export const store = configureStore({
```

```

    reducer: { counter: counterSlice.reducer }
  });

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

```

Redux Toolkit dùng Immer bên trong reducer logic, nên cú pháp nhìn giống mutation (`state.value += 1`) nhưng tạo immutable update an toàn theo cơ chế của toolkit.

30.3 Typed hooks

```

import { useDispatch, useSelector } from "react-redux";

export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppState = useSelector.withTypes<RootState>();

```

30.4 Async thunk

```

import { createAsyncThunk, createSlice } from "@reduxjs/toolkit";

type User = { id: number; name: string };

export const fetchUsers = createAsyncThunk<User[]>(
  "users/fetchUsers",
  async () => {
    const res = await fetch("/api/users");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    return res.json();
  }
);

const usersSlice = createSlice({
  name: "users",
  initialState: { data: [] as User[], status: "idle" as "idle" | "loading" | "success" | "error" },
  reducers: {},
  extraReducers(builder) {
    builder
      .addCase(fetchUsers.pending, state => { state.status = "loading"; })
      .addCase(fetchUsers.fulfilled, (state, action) => {
        state.status = "success";
        state.data = action.payload;
      })
      .addCase(fetchUsers.rejected, state => { state.status = "error"; });
  }
});

```

30.5 Middleware là gì?

Redux middleware đứng giữa `dispatch(action)` và reducer, có thể quan sát/biến đổi flow: logging, analytics, async handling, error reporting... Middleware không nên tùy tiện chứa toàn bộ business logic.

```

import type { Middleware } from "@reduxjs/toolkit";

const logger: Middleware = storeApi => next => action => {
  console.log("before", storeApi.getState());
  const result = next(action);
  console.log("after", storeApi.getState());
  return result;
};

const store = configureStore({
  reducer: rootReducer,
  middleware: getDefaultMiddleware =>
    getDefaultMiddleware().concat(logger)
});

```

30.6 Redux flow

33. UI dispatch action.
34. Middleware có thể xử lý/quan sát action.
35. Reducer nhận state hiện tại + action và tạo state tiếp theo.
36. Store cập nhật state.
37. Subscribed React components đọc state mới và render lại khi selector result thay đổi.

30.7 Normalized state

Với collection lớn, lưu entity theo ID giúp update/query hiệu quả và tránh duplication. Redux Toolkit có helper cho entity adapter, nhưng quan trọng hơn là hiểu nguyên tắc normalized state.

```
type UsersState = {
  ids: number[];
  entities: Record<number, User | undefined>;
};
```

31. Kiến trúc ứng dụng, Error Handling và Best Practices

31.1 Gợi ý structure theo feature

```
src/
  app/
    store.ts
    router.tsx
  features/
    auth/
      api.ts
      components/
      hooks.ts
      types.ts
    users/
      api.ts
      components/
      pages/
      types.ts
  shared/
    components/
    hooks/
    lib/
    types/
```

Feature-based structure thường scale tốt hơn việc có một thư mục `components/` khổng lồ chứa mọi thứ không phân biệt domain.

31.2 Error boundary

Error boundary bắt lỗi render/lifecycle trong subtree và hiển thị fallback thay vì làm hỏng toàn bộ UI. Nó không thay thế try/catch cho event handler hoặc network request. Cơ chế cụ thể phụ thuộc cách bạn xây app/framework.

31.3 Derived state

```
// Tốt: tính từ source of truth
const visibleTodos = todos.filter(todo => filter === "all" || todo.status === filter);

// Cảnh thận: lưu visibleTodos thành state riêng có thể bị lệch với todos/filter.
```

31.4 State placement

Loại state	Nơi phù hợp
UI local: modal open, input draft	useState trong component gần nhất
Complex local transition	useReducer
Shared app client state	Context hoặc Redux tùy độ phức tạp
Server data/cache	Data-fetching layer / query cache / framework data API
URL state	Router params/search params khi cần share/bookmark/back-forward

31.5 Security mindset

- Không tin dữ liệu client. Backend phải authorize/validate.
- Không đưa secret vào frontend bundle.
- Escape/render text theo cách React mặc định; cực kỳ cẩn thận với HTML raw.
- Hiểu XSS, CSRF, CORS, cookie flags và token lifecycle ở mức phù hợp vai trò.
- Không log password/token/PII tùy tiện.

31.6 Testing mindset

Ưu tiên test behavior người dùng nhìn thấy: render, interaction, state transition, request boundary. Unit test phù hợp pure function/reducer/validator; integration test phù hợp flow component + router + API mock. Không nên test implementation detail quá chặt.

32. TypeScript + React: các pattern nên nhớ

32.1 Props union để loại trạng thái không hợp lệ

```
type ButtonProps =
  | { kind: "link"; href: string; onClick?: never }
  | { kind: "button"; onClick: () => void; href?: never };

function Action(props: ButtonProps) {
  if (props.kind === "link") {
    return <a href={props.href}>Open</a>;
  }
  return <button onClick={props.onClick}>Run</button>;
}
```

32.2 API response + discriminated union

```
type RemoteData<T> =
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };

function UserPanel({ state }: { state: RemoteData<User[]> }) {
  if (state.status === "loading") return <Spinner />;
  if (state.status === "error") return <ErrorMessage text={state.error} />;
  return <UserList users={state.data} />;
}
```

32.3 Event callback contract

```
type ModalProps = {
  open: boolean;
  onClose: () => void;
  onConfirm: (reason: string) => Promise<void>;
};
```

32.4 Không lạm dụng React.FC

Bạn có thể type props trực tiếp trên function component. Điều này thường rõ ràng và kiểm soát `children` explicit hơn. Quan trọng là team convention nhất quán, không phải một quy tắc tuyệt đối.

33. Cheat Sheet - TypeScript

Mục tiêu	Cú pháp/pattern
Union	A B
Intersection	A & B
Optional property	name?: string
Readonly	readonly id: number
Generic	function f<T>(x: T): T
Constraint	T extends Base
Key union	keyof T
Type from value	typeof value
Indexed access	T[K]
Type guard	value is User
Mapped type	{ [K in keyof T]: ... }
Conditional type	T extends U ? X : Y
Infer	T extends Promise<infer U> ? U : T
Exact literal preservation	as const
Validate while preserving inference	satisfies SomeType

TypeScript mini-checklist trước khi commit

- Có `any` nào có thể thay bằng `unknown` hoặc type thật không?
- Dữ liệu external đã validate chưa?
- Có assertion `as` nào chỉ đang che lỗi thiết kế không?
- Union có thể dùng discriminant để narrow rõ hơn không?
- State/type có cho phép trạng thái vô lý không?
- Generic có thật sự tăng reuse hay chỉ tăng độ khó?

34. Cheat Sheet - ReactJS

Nhu cầu	Công cụ/pattern
Local state	useState
Complex local state	useReducer
External synchronization	useEffect
DOM/reference mutable	useRef
Shared contextual value	Context
Reusable stateful logic	Custom hook
Derived expensive calculation	useMemo khi có lý do
Stable callback cho optimization	useCallback khi có lý do
Navigation/URL state	Router
Complex shared client state	Redux Toolkit nếu cần
Server request	fetch/API layer/query abstraction

React mini-checklist trước khi commit

- Component có làm quá nhiều việc không?
- State có thể đặt gần nơi sử dụng hơn không?
- Derived data có bị lưu trùng thành state không?
- Effect này có thật sự đồng bộ external system không?
- Effect có cleanup khi cần không?
- List key có ổn định không?
- Form có label/error/accessibility cơ bản không?
- Loading/error/empty state của API đã xử lý chưa?
- Authorization có đang bị nhầm với UI guard không?
- Có memoization nào thêm vào mà không đo/không cần không?

35. Luồng học đề xuất

Giai đoạn	Nội dung trọng tâm	Bạn nên làm được
1. Nền tảng JS/ES6+	scope, array methods, destructuring, modules, async/await	Đọc/viết JS hiện đại không vướng cú pháp.
2. TypeScript core	types, function, interface, narrowing, generics	Type model cho object/API/component props.
3. React core	JSX, component, props, state, events, effects	Làm app CRUD nhỏ.
4. Form/API/router	forms, validation, REST, router	App nhiều trang, gọi backend ổn định.
5. Shared state	Context, reducer, Redux Toolkit	Quản lý flow phức tạp có cấu trúc.
6. Production mindset	SSR, performance, security, testing	Biết trade-off và tránh anti-pattern phổ biến.

Bài tập tổng hợp đề xuất

- Xây Todo app bằng React + TypeScript: create/update/delete/filter, localStorage.
- Xây User Management: list, detail, create/edit form, validation, loading/error/empty state.
- Thêm router: `/users`, `/users/:id`, `/users/new`.
- Tách API layer và validate response.
- Thêm auth mock + protected UI; nhớ backend mới là nơi authorize thật.
- Khi state/phối hợp nhiều feature đủ phức tạp, chuyển phần phù hợp sang Redux Toolkit.

36. Câu hỏi tự kiểm tra nhanh

44. Vì sao `unknown` an toàn hơn `any`?
45. Sự khác nhau giữa `interface` và `type` trong các tình huống phổ biến?
46. Khi nào dùng generic constraint?
47. Discriminated union giúp loại lỗi state như thế nào?
48. Tại sao type assertion không phải runtime validation?
49. Trong React, vì sao không nên mutate state?
50. Khi nào `useEffect` là không cần thiết?
51. Vì sao key bằng index có thể gây bug khi reorder danh sách?
52. Controlled input là gì?
53. Context khác Redux ở mục tiêu sử dụng như thế nào?
54. Client-side protected route có phải security boundary không?
55. SSR khác CSR ở đâu và hydration là gì?
56. Redux middleware nằm ở đâu trong dispatch flow?
57. Khi nào không nên dùng Redux?
58. Bạn sẽ đặt API loading/error state ở đâu và model type như thế nào?

KẾT LUẬN

TypeScript và React bổ trợ cho nhau rất tốt: TypeScript làm contract của data/component rõ ràng, còn React cung cấp mô hình khai báo để tổ chức UI theo component và state. Nền tảng quan trọng nhất không phải nhớ mọi API, mà là hiểu data flow, state ownership, side-effect boundary, type narrowing và cách thiết kế các trạng thái hợp lệ.

Nguyên tắc cuối: Type tốt + component nhỏ vừa đủ + state đặt đúng chỗ + effect có mục đích + validation tại boundary + error/loading rõ ràng = nền tảng rất vững cho phần lớn ứng dụng React + TypeScript.